

Etapas en URP

Marco Antonio Gómez Martín
Iluminación y Materiales
Grado en Desarrollo de Videojuegos
2025/2026



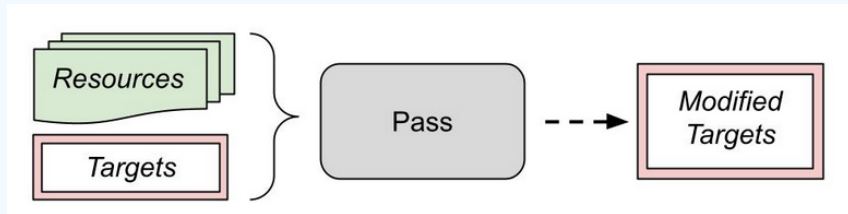
Facultad
de
Informática



UNIVERSIDAD COMPLUTENSE
MADRID

Recordatorio - Render graphs - Pasadas

El renderizado se divide en pasadas



Recordatorio - Render graphs - Fases

- **Setup:** se declaran las pasadas y los recursos de entrada y salida que utilizan. Los recursos son “virtuales”.
- **Compile:** se determina el ciclo de vida de los recursos, se eliminan pasadas no relevantes, se hacen las reservas de recursos necesarias.
- **Execute:** se ejecutan las pasadas.

Recordatorio - SRP Render pipeline a vista de pájaro

```
void RenderCamera(ScriptableRenderContext context,
                  Camera cameraToRender) {

    ...

    try
    {
        myRenderGraph.BeginRecording(rgParams);
        // Add render passes here.
        myRenderGraph.EndRecordingAndExecute();
    }
    catch (Exception e)
    {
        if (renderGraph.ResetGraphAndLogException(e))
            throw;
    }

    ...

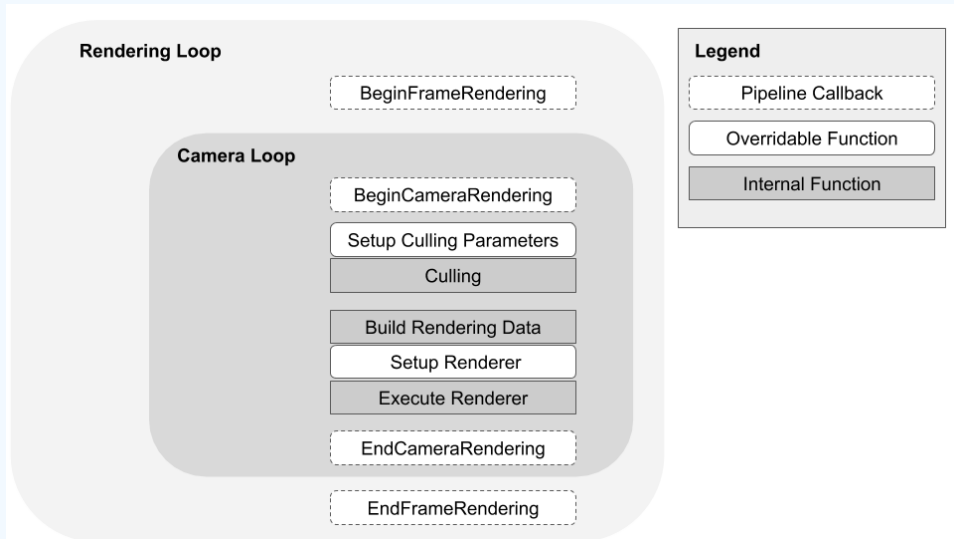
}
```

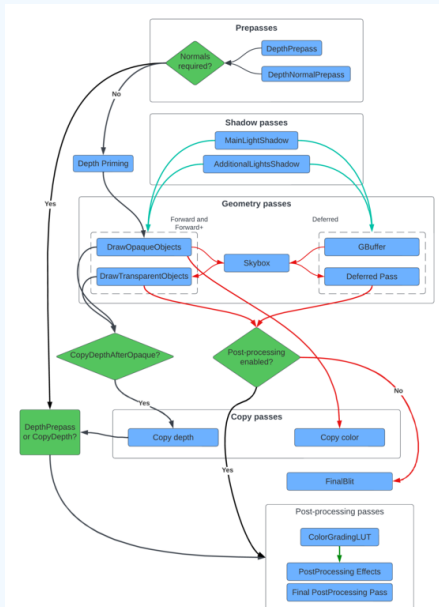
Recordatorio - SRP Render pipeline a vista de pájaro

- `// Add render passes here` significa "añadir nodos al render graph".
- Los nodos se añaden en el orden en el que se ejecutarán.

¿Y en URP cómo es?

Universal Render Pipeline





Etapas en URP - Injection points

- Shadows
- PrePasses
- GBuffer
- DeferredLights
- Opaques
- Skybox
- Transparents
- PostProcessing

Nuevas pasadas en URP

- Las *Render features* añadidas a los *renderers* son las responsables de añadir nuevas pasadas.
- Las pasadas se implementan heredando de `ScriptableRenderPass`.
- En la implementación de la pasada, se añade al *render graph* los "nodos" necesarios para el dibujado de la pasada.
- Para cada nodo se definen sus recursos de entrada y salida y la función que define las instrucciones de renderizado de ese nodo.
- Unity da algunas formas cómodas de construir/registrarse nodos de funcionalidad sencilla.

Nada de esto se entiende hasta que no veamos un ejemplo...

Nuevas pasadas en URP

Aunque el API es versatil, es bastante confuso.

- El término "pasada" se puede referir a muchas cosas.
- Las Render Features *no* son pasadas.
- Las Render Features que añadamos *no* se pintan en el orden en el que las pongamos.
- Las `ScriptableRenderPass` pueden añadir varias pasadas/nodos al render graph.

La clase `ScriptableRenderPass` representa una “pasada”:

- Signifique lo que signifique lo de “pasada”...
- El *Renderer* de la cámara tiene la lista de todas las pasadas que hay que utilizar.
- Implementa un único método `RecordRenderGraph` que recibe el render graph e información del contexto de renderizado.
- Tiene un atributo que le dice al renderer en qué momento hay que llamar a la función anterior.
- Añade al render graph el nodo (o nodos) que hay que ejecutar (de ahí que decir que representa una “pasada” pueda ser confuso).
- Para cada nodo se configuran sus recursos y función de renderizado.

Nuevas pasadas en URP

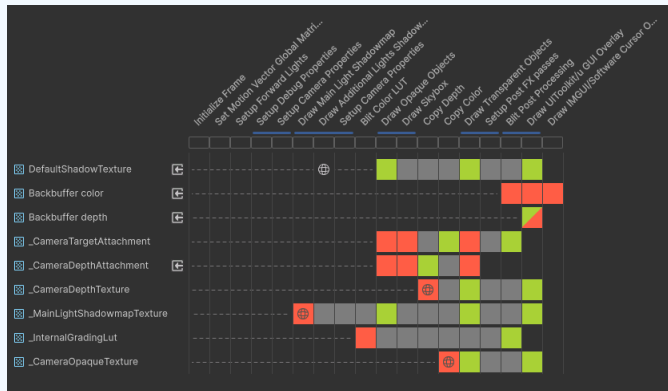
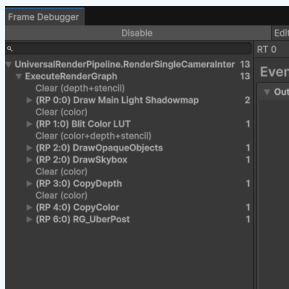


Mejor con ejemplos...

Punto de partida

- Proyecto de URP.
- Escena con algo de geometría.
- Eliminadas fases de postprocesado.
 - En el renderer
 - En el volúmen

Punto de partida



Ejemplo 1: CopyPass

- Pasada que copia el color buffer a una textura auxiliar.
- Permitiremos configurar en qué momento se hace.
- Copia vs. blit.
- Usaremos `renderGraph.AddCopyPass(origen, destino, nombrePasada)` (un método de extensión en `UnityEngine.Rendering.RenderGraphModule.Util`), que añade un nodo de "copia" al grafo.

Ejemplo 1: CopyPass

```
using UnityEngine.Rendering;
using UnityEngine.Rendering.RenderGraphModule;
using UnityEngine.Rendering.RenderGraphModule.Util;
using UnityEngine.Rendering.Universal;

class CopyPass : ScriptableRenderPass {

    public CopyPass()
    {
        requiresIntermediateTexture = true;
    }

    public override void RecordRenderGraph(RenderGraph renderGraph,
                                           ContextContainer frameData) {
        ...
    }
}
```


Ejemplo 1: CopyPass

```
...

public override void RecordRenderGraph(RenderGraph renderGraph,
                                       ContextContainer frameData) {
    // Creamos el recurso/textura donde copiaremos
    UniversalResourceData resourceData =
        frameData.Get<UniversalResourceData>();
    var source = resourceData.activeColorTexture;
    var destinationDesc = renderGraph.GetTextureDesc(source);
    destinationDesc.name = $"CameraColor-{passName}";
    TextureHandle destination = renderGraph.CreateTexture(destinationDesc);

    // Creamos el nodo
    renderGraph.AddCopyPass(resourceData.activeColorTexture,
                           destination, passName);
}

...
```

Ejemplo 1: CopyPass

```
using UnityEngine.Rendering.Universal;

public class CopyPassFeature : ScriptableRendererFeature {
    private CopyPass m_ScriptablePass;

    // Llamado al incorporar, activar y desactivar la feature en el
    // editor, y al cambiar sus propiedades (si hay)
    public override void Create() {
        m_ScriptablePass = new CopyPass();

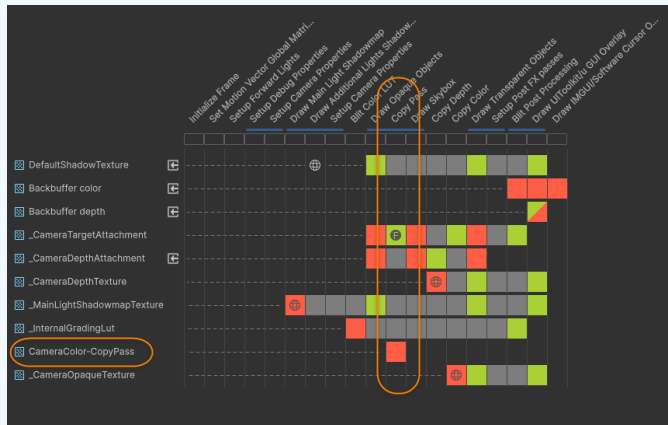
        m_ScriptablePass.renderPassEvent =
            RenderPassEvent.AfterRenderingOpaques;
    }

    public override void AddRenderPasses(ScriptableRenderer renderer,
                                         ref RenderingData renderingData) {
        renderer.EnqueuePass(m_ScriptablePass);
    }
}
```

Ejemplo 1: CopyPass

Resultado

UniversalRenderPipeline.RenderSingleCameraInter	15
▼ ExecuteRenderGraph	15
Clear (depth+stencil)	
▶ (RP 0:0) Draw Main Light Shadowmap	2
Clear (color)	
▶ (RP 1:0) Blit Color LUT	1
Clear (color+depth+stencil)	
▶ (RP 2:0) DrawOpaqueObjects	1
Clear (color)	
▼ (RP 2:1) CopyPass	1
Draw Procedural	
▶ (RP 2:2) DrawSkybox	1
Clear (color)	
▶ (RP 3:0) CopyDepth	1
Clear (color)	
▶ (RP 4:0) CopyColor	1
▼ (RP 6:0) RG_UberPost	1
Draw Procedural	



Ejemplo 2: Textura de color plano

- Pasada que crea una textura auxiliar (que nadie usará).
- La textura se rellena con un color plano.
- El color se configura desde la Render Feature.

Ejemplo 2: Textura de color plano

```
using UnityEngine;
using UnityEngine.Rendering.Universal;

public class CreateColorTextureFeature : ScriptableRendererFeature {

    [SerializeField] private Color colorRelleno;

    CreateColorTexturePass customPass;

    public override void Create() {
        customPass = new CreateColorTexturePass();
        customPass.renderPassEvent = RenderPassEvent.AfterRenderingOpaques;
    }

    public override void AddRenderPasses(ScriptableRenderer renderer,
                                         ref RenderingData renderingData) {
        customPass.Setup(colorRelleno);
        renderer.EnqueuePass(customPass);
    }
}
```

Ejemplo 2: Textura de color plano

```
using UnityEngine;
using UnityEngine.Rendering.Universal;
using UnityEngine.Rendering.RenderGraphModule;
using UnityEngine.Rendering;

class CreateColorTexturePass : ScriptableRenderPass {

    Color _color = Color.magenta;

    public void Setup(Color color) {
        _color = color;
    }

    class PassData {
        public Color color;
    }

    // ...
    static void ExecutePass(PassData data, RasterGraphContext context) {
        context.cmd.ClearRenderTarget(true, true, data.color);
    }
}
```

Ejemplo 2: Textura de color plano

```
public override void RecordRenderGraph(RenderGraph renderGraph,
                                       ContextContainer frameData) {
    using (var builder = renderGraph.AddRasterRenderPass<PassData>(
        "Create color texture", out var passData)) {

        UniversalResourceData resourceData = frameData.Get<UniversalResourceData>();

        // Textura temporal como render target
        var textureDesc = renderGraph.GetTextureDesc(resourceData.activeColorTexture);
        TextureHandle texture = renderGraph.CreateTexture(textureDesc);
        textureDesc.name = "TexturaPlana";
        builder.SetRenderAttachment(texture, 0, AccessFlags.Write);

        // Parametro y funcion render
        passData.color = _color;
        builder.SetRenderFunc(static (PassData data, RasterGraphContext context) =>
            ExecutePass(data, context));
    }
}
```

Ejemplo 2: Textura de color plano

Para ver su ejecución, hay que *forzar* la aparición del nodo, de forma que en la compilación no se elimine por no ser de utilidad

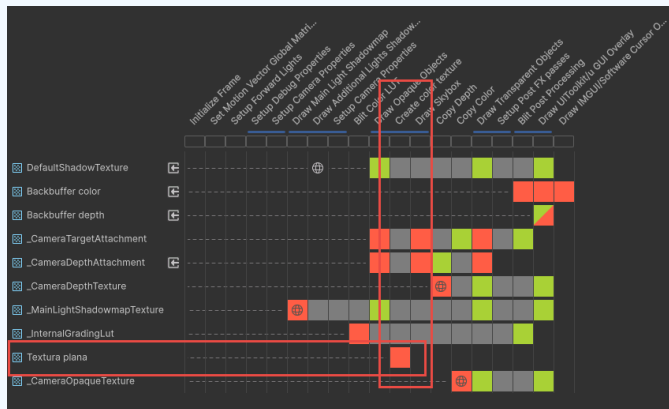
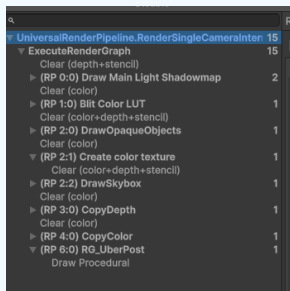
```
using (var builder = renderGraph.AddRasterRenderPass<PassData>(
    "Create color texture", out var passData)) {
    // ...

    builder.AllowPassCulling(false);

    // ...
}
```


Ejemplo 2: Textura de color plano

Resultado



Etapas en URP

¿Preguntas?



Facultad
de
Informática



UNIVERSIDAD COMPLUTENSE
MADRID